

DATABRICKS MILESTONE 2

Important Topics & Concepts
Quick Revision Notes

Java • Scala • Spark RDDs • DataFrames • Advanced Spark • Streaming • JDBC •
Concurrency

Purpose: A simple revision guide based on the uploaded Milestone 2 Preparation Handbook. It focuses on important concepts, comparisons, common traps and scenario cues.

Source: Databricks Milestone 2 – Trainee Preparation & Practice Notes (39 pages).

1. What to Prioritize

The handbook says the assessment mixes concept recall, code interpretation, basic application and scenario reasoning. Practice explaining *why* an option is correct, not only memorizing it.

- **HIGH:** Java fundamentals/OOP, Scala basics, RDD transformations/actions, pair RDDs, persistence, DataFrame/schema reasoning.
- **MEDIUM:** Java I/O/APIs, Scala file handling, functional transformations, execution plans, UDFs, JDBC integration.
- **STRETCH:** Streaming recovery/windows, concurrency, deadlocks/races, CompletableFuture, concurrent collections, HikariCP, JDBC rollback.

Priority	Topics
HIGH	Java core/OOP • Scala basics • RDDs • Pair RDDs • Persistence • DataFrames/schemas
MEDIUM	Java I/O/APIs • Scala file handling • Functional transformations • Advanced Spark/JDBC
STRETCH	Streaming • Concurrency • Deadlocks/races • Transactions/pooling

Do not overprepare on deep JVM GC/bytecode internals, lock-free concurrency, advanced JDBC isolation/pool tuning, Spark scheduler/Catalyst internals, or complex watermark edge cases.

2. Java Fundamentals & Collections

JDK / JRE / JVM

Concept: Java source is compiled into bytecode; the JVM executes the bytecode. The JDK contains development tools such as javac.

Remember: Compile → javac/JDK. Execute bytecode → JVM.

Common trap: Do not say the JVM is the Java compiler.

Stack vs Heap

Concept: A method call has a stack frame. Created objects and arrays are generally allocated on the heap and referenced by variables.

Remember: Execution/local method context → stack; objects → heap.

Wrapper Classes & Autoboxing

Concept: Wrappers represent primitive values as objects, e.g. Integer for int. Autoboxing is primitive → wrapper; unboxing is wrapper → primitive.

Remember: List uses a reference type.

Static vs Instance

Concept: Static members belong to the class; instance members belong to each object.

Remember: static ≠ constant.

List / Set / Map

Concept: List is ordered and permits duplicates; Set models uniqueness; Map stores key-value associations.

Remember: Ask: order? duplicates? key-value lookup?

HashSet: equals() + hashCode()

Concept: HashSet uses both to determine logical uniqueness.

Remember: Custom objects should implement them consistently.

Common trap: Different objects can still be logically equal.

HashMap Keys

Concept: Keys identify mappings; inserting an equal existing key replaces its previous value.

Remember: Values may repeat; key identity drives lookup.

3. Java Core + OOP Must-Know

Generics

Concept: Generics provide compile-time type safety and reusable parameterized code.

Remember: List should contain String values.

Wildcards

Concept: ? extends Number means Number or an unknown subtype of Number.

Remember: extends = upper bound.

Iterator / ListIterator

Concept: Iterator traverses and can safely remove the last returned element. ListIterator can move forward and backward through a List.

Remember: Use iterator.remove() for coordinated removal during iteration.

Numeric Promotion

Concept: byte and short are promoted to int in arithmetic.

Remember: byte + byte → int.

Common trap: Do not assume the result remains byte.

Short-Circuit && / ||

Concept: && stops when the left side is false; || stops when the left side is true.

Remember: Useful for guarded/null-check conditions.

Operator Precedence

Concept: Operators are grouped according to precedence before evaluation.

Remember: $2 + 3 * 4 \rightarrow 14$.

Constructors + this()/super()

Concept: this() calls another constructor in the same class; super() calls a parent constructor.

Remember: An explicit constructor call must be first; parent constructor runs before child body.

Overloading vs Overriding

Concept: Overloading = same name/different parameters/compile-time choice. Overriding = subclass replacement/runtime dispatch.

Remember: A changed return type alone does not overload a method.

Runtime Polymorphism

Concept: A parent/interface reference can point to a child implementation and invoke the actual object's overridden method.

Remember: Parent reference + child object → child implementation.

4. Java OOP – High-Value Concepts

Inheritance vs Composition

Concept: Inheritance models **is-a**; composition models **has-a**.

Remember: Dog is an Animal → inheritance. Car has an Engine → composition.

Interface

Concept: Defines a contract implemented by classes; supports interchangeable implementations.

Remember: An interface is not instantiated directly.

Abstract Class

Concept: Can contain shared state/implemented behavior plus abstract methods and cannot be instantiated directly.

Remember: Useful for closely related subclasses sharing code/state.

Encapsulation

Concept: Hide internal state and expose controlled operations, commonly through private fields and public methods.

Remember: private state + controlled public behavior.

final

Concept: final variable → no reassignment; final method → cannot be overridden; final class → cannot be inherited.

Remember: final does not automatically make referenced objects immutable.

this vs super

Concept: this refers to the current object; super refers to parent behavior/constructor.

Remember: this.name = name; | super.display();

SRP

Concept: A class should have one primary reason to change.

Remember: Avoid mixing unrelated responsibilities in one class.

5. Java File I/O, APIs & Concurrency

Byte vs Character Streams

Concept: Byte streams handle raw bytes; character-oriented APIs are suited to text.

Remember: Image/PDF → byte-oriented. CSV/log text → character-oriented.

FileOutputStream vs PrintWriter

Concept: FileOutputStream writes bytes; PrintWriter provides convenient text output such as print/println/printf.

Try-with-Resources

Concept: Automatically closes AutoCloseable resources when the block exits, even on exceptions.

Remember: Use for files and JDBC resources.

Java Streams API

Concept: Declarative collection processing with filter, map, sorted, reduce and collect/toList.

Remember: Do not confuse Java Stream with InputStream or Spark Streaming.

Regex: matches() vs find()

Concept: matches() attempts to match the entire input; find() searches for a matching subsequence.

Remember: Whole-value validation → matches(); search inside text → find().

java.time

Concept: LocalDate = date; LocalDateTime = date + time; ZonedDateTime = date + time + zone.

HttpClient

Concept: HttpClient sends HTTP requests; HttpRequest describes the request; HttpResponse contains the response.

Remember: Basic flow: request → send → response.

transient

Concept: A transient field is skipped by default Java serialization.

Remember: It is not access control or database persistence.

Race Condition

Concept: Multiple threads access shared mutable state and the result depends on timing/interleaving.

Remember: Shared state + non-atomic read-modify-write is the warning sign.

Deadlock

Concept: Threads wait indefinitely for locks held by each other.

Remember: Consistent global lock ordering reduces circular-wait risk.

ConcurrentHashMap

Concept: Map designed for safe concurrent access/updates.

Remember: Choose it when multiple threads need concurrent map access.

CopyOnWriteArrayList

Concept: Copies internal data on modification, making iteration safe but writes expensive.

Remember: Best fit: many reads + rare writes.

CompletableFuture

Concept: Represents an asynchronous computation that can be chained or combined.

Remember: Think asynchronous work + result coordination.

6. JDBC & Database Concepts

Transaction / Commit / Rollback

Concept: A transaction groups related changes so they succeed or fail as a unit.

Remember: `commit()` makes changes permanent; `rollback()` undoes uncommitted work.

Example:

```
setAutoCommit(false)
update A
update B
commit()
// failure → rollback()
```

Connection Pooling / HikariCP

Concept: Reuses database connections instead of creating a physical connection for every operation.

Remember: Borrow → use → close/return to pool.

Spark JDBC Parallel Reads

Concept: `partitionColumn`, `lowerBound`, `upperBound` and `numPartitions` support parallel JDBC reads when appropriate partitioning is available.

Remember: `numPartitions` also limits concurrent JDBC partitions/connections.

Common trap: Avoid excessive partitions beyond database capacity.

Scenario cues

- Two related DB updates must both succeed → transaction + commit/rollback.
- Repeated connection creation is expensive → connection pooling.
- Large/slow JDBC read with suitable partition column → partitioned JDBC read.
- Too many database connections → reconsider `numPartitions`.

7. Scala Basics

val vs var

Concept: val cannot be rebound; var can be reassigned.

Remember: Prefer val in functional style.

Type Inference

Concept: Scala can infer many variable/expression types.

Remember: val name = "Asha" → String; val score = 95 → Int.

Tuples

Concept: Group a fixed number of values, potentially with different types.

Remember: Use `_1/_2` or destructuring.

Array vs List

Concept: Array is indexed and mutable; List is immutable.

Remember: Array can update an element; immutable List operations return new Lists.

::

Concept: Prepends an element to an immutable List and returns a new List.

Remember: `1 :: List(2,3) → List(1,2,3)`.

Pattern Matching

Concept: match checks ordered case patterns and can destructure values.

Remember: case `_` is the wildcard fallback.

Pattern Guards

Concept: Adds an if-condition to a case; both pattern and guard must match.

Lambda

Concept: Anonymous function value, e.g. `x => x * 2`.

8. Scala Functional Transformations

map

Concept: Transforms each element and returns one result per input element.

Remember: One input → one output.

filter

Concept: Keeps elements satisfying a predicate.

flatMap

Concept: Maps and then flattens nested results into one collection.

Remember: Use when one input can produce zero/one/many outputs.

Higher-Order Functions

Concept: Accept or return functions. A common predicate type is $A \Rightarrow \text{Boolean}$.

Pure Functions

Concept: Output depends only on explicit inputs and there are no hidden side effects.

Remember: Easier to reason about and test.

Immutability

Concept: Derive new values rather than changing shared state in place.

Remember: Supports safer reasoning in concurrent/distributed processing.

Operation	Meaning	Shape
map	Transform every element	$1 \rightarrow 1$
filter	Keep matching elements	0 or 1 per input
flatMap	Transform + flatten	0 / 1 / many

9. Spark RDDs – HIGH PRIORITY

The handbook assigns 16 MCQs to RDDs and calls this one of the most important chapters.

RDD Creation

Concept: `parallelize` converts a local collection into an RDD; `textFile` reads an external text source.

Remember: Local collection → `parallelize`. External text → `textFile`.

Transformations

Concept: `map`, `filter`, `flatMap`, `reduceByKey` etc. define new RDDs and are lazy.

Remember: They build lineage; they do not themselves trigger full execution.

Actions

Concept: Actions trigger execution and return/write results.

Remember: `count()`, `collect()`, `take()`, `reduce()`, `save` operations.

Lazy Evaluation

Concept: Spark waits until an action is called before executing required transformation work.

Remember: Allows planning and avoids unused work.

Narrow vs Wide

Concept: Narrow transformations avoid broad redistribution; many group/repartition-style operations require shuffle.

Remember: `map/filter` are typical narrow transformations.

`collect()`

Concept: Brings the entire RDD to the driver.

Remember: Large RDD + `collect()` can exhaust driver memory.

Lineage

Concept: Transformation dependency chain used to derive an RDD.

Remember: Supports recomputation after partition loss.

Pair RDD + `reduceByKey`

Concept: Key-value tuples can be aggregated per key using `reduceByKey`.

Remember: Prefer distributed per-key aggregation instead of collecting everything to the driver.

Join

Concept: Combines Pair RDDs using a common key.

Remember: Key-based joins can cause shuffle.

`persist/cache`

Concept: Keeps computed partitions for reuse.

Remember: Use when an expensive RDD is reused; do not cache everything.

Storage Levels

Concept: Control whether persisted partitions use memory, disk, serialized forms, etc.

Remember: Know `MEMORY_ONLY` vs `MEMORY_AND_DISK` conceptually.

mapPartitions

Concept: Runs a function once per partition and provides an iterator.

Remember: Useful when expensive initialization can be shared within a partition.

10. Spark DataFrames – HIGH PRIORITY

RDD vs DataFrame

Concept: RDD is low-level distributed data. DataFrame is schema-aware, column-oriented and easier for Spark to optimize for structured operations.

Remember: Structured tabular/JSON/Parquet + SQL-style processing → DataFrame.

Schema Inference vs Explicit Schema

Concept: Inference guesses types from data; explicit schema declares expected names/types/nullability.

Remember: Explicit schema gives predictable types and consistency.

Malformed Values

Concept: A declared schema does not magically make malformed text valid.

Remember: Reader mode/options determine handling.

Structured Expressions

Concept: col(), filter(), groupBy() and built-ins preserve structure/semantics Spark can analyze and optimize.

Remember: Prefer built-ins when possible.

Batch Spark Flow

Concept: Basic Scala Spark application: SparkSession → read → transform → action/write.

Remember: Know the pipeline order.

11. Advanced Spark – Execution Plans, UDFs & Sources

Logical vs Physical Plan

Concept: Logical plan describes what the query means; physical plan describes how Spark will execute it.

Remember: `df.explain()` helps inspect plans.

Exchange / Shuffle

Concept: Exchange commonly indicates repartitioning/data redistribution required by some joins and aggregations.

Remember: Shuffle is a performance consideration because it moves data.

Built-in Functions vs UDFs

Concept: Built-ins expose semantics Spark can optimize. UDFs are for custom logic not conveniently expressed with built-ins.

Remember: First ask: can a built-in function solve it?

Filter Before Expensive UDF

Concept: Apply selective built-in filters before expensive custom logic when logically valid.

Remember: Fewer rows reach the expensive UDF.

JDBC Partition Read

Concept: Partitioned reads can use `partitionColumn`, `bounds` and `numPartitions` for parallelism.

Remember: `numPartitions` controls read partitions/concurrent JDBC work; do not oversubscribe the database.

Parquet Pruning

Concept: Parquet is columnar. Selecting needed columns enables column pruning; filtering partition columns can enable partition pruning.

Remember: Both reduce unnecessary I/O.

JSON vs Parquet

Concept: JSON is flexible/semi-structured and convenient for raw ingestion; Parquet is columnar and efficient for downstream analytics.

Remember: Choose based on workload/stage.

12. Spark Streaming & Fault Tolerance – STRETCH

DStreams vs Structured Streaming

Concept: DStreams are the older RDD-based streaming abstraction; Structured Streaming uses DataFrame/Dataset structured APIs.

Remember: New DataFrame-style streaming → Structured Streaming.

File Streaming

Concept: Structured Streaming can process newly arriving files incrementally.

Remember: Checkpointing helps track progress across restarts.

Socket Source

Concept: Socket sources are easy for demonstrations but are not durable production sources.

Remember: Do not use an ephemeral socket source as the durability mechanism for production recovery.

Checkpointing

Concept: Stores progress/state metadata needed for a compatible streaming query to resume after failure.

Remember: Use durable storage; checkpoint is not the same as RDD cache/persist.

Windows

Concept: A window groups recent streaming records over a time interval; slide controls how frequently the result advances.

Remember: 30-minute window updated every 5 minutes.

Late Data & Watermarks

Concept: Event-time records can arrive late. Watermarks express how long the engine should wait/retain state for late events.

Remember: Longer tolerance accepts later data but may retain state longer.

Streaming scenario cues

- Restart without starting from zero → checkpoint.
- Rolling recent-period result → window.
- Late events / bounded state → watermark.
- New DataFrame-style streaming app → Structured Streaming.

13. Rapid Comparison Sheet

Pair	Key distinction
Overloading vs Overriding	Overloading: same name/different parameters/compile time. Overriding: subclass replacement/runtime dispatch.
List vs Set vs Map	List: ordered/duplicates. Set: uniqueness. Map: key-value.
val vs var	val cannot be rebound. var can be reassigned.
Array vs List	Array mutable/indexed. List immutable.
map vs flatMap	map = one result per input. flatMap = expand + flatten.
Transformation vs Action	Transformation builds lazy lineage. Action triggers execution.
RDD vs DataFrame	RDD low-level. DataFrame schema-aware/optimizer-friendly.
persist/cache vs checkpoint	Persist speeds reuse. Checkpoint stores recovery/progress/state metadata.
DStream vs Structured Streaming	DStream legacy RDD-based. Structured Streaming uses structured APIs.
matches vs find	matches = whole input. find = subsequence search.
commit vs rollback	commit makes changes permanent. rollback undoes uncommitted work.
Race vs Deadlock	Race = timing-dependent wrong result. Deadlock = indefinite waiting.

14. Final Must-Know Checklist

- ■ JVM, bytecode, stack/heap
- ■ Collections, generics, iterators, operators
- ■ Constructors, overloading/overriding, polymorphism
- ■ this, super, final, interfaces, abstract classes, composition, encapsulation, SRP
- ■ Java I/O, try-with-resources, Streams, regex, date/time, HttpClient, transient
- ■ Scala val/var, tuples, Array/List, match, guards, lambdas
- ■ map/filter/flatMap, higher-order and pure functions
- ■ RDD transformations/actions, lazy evaluation, lineage, pair RDDs, reduceByKey, join
- ■ collect(), persist/cache, storage levels, mapPartitions
- ■ DataFrame vs RDD, schemas, structured expressions
- ■ explain(), Exchange/shuffle, UDF vs built-in, JDBC partitions, Parquet pruning
- ■ Structured Streaming, checkpointing, windows, watermarks
- ■ Race/deadlock, concurrent collections, CompletableFuture
- ■ JDBC transactions and connection pooling

15. 48-Hour Revision Order

Block	Focus
Day 1 – Block 1	Java fundamentals + collections + OOP
Day 1 – Block 2	Java I/O/APIs + short runnable exercises
Day 1 – Block 3	Scala basics + functional transformations
Day 2 – Block 1	RDDs + pair RDDs + persistence + lineage
Day 2 – Block 2	DataFrames + schemas + Advanced Spark scenarios
Day 2 – Block 3	Streaming + JDBC/concurrency stretch + self-check

Final advice: Do not memorize answer positions or wording. Know the underlying concept well enough to explain why the correct option solves the scenario and why the alternatives do not.

This quick guide stays at the assessment depth described in the uploaded handbook and does not expand into deep production-engineering internals.